

To install fastapi with uvicorn:, run command: *pip install fastapi[all]*

```
(.venv) C:\Tests\Web\FastAPI-Code\Cookbook>uvicorn --version
Running uvicorn 0.35.0 with CPython 3.13.3 on Windows
```

There is no unique and perfect structure for a FastAPI project; however, a common approach is to divide your project into several key directories:

- `/src`: This is where your primary application code lives. Inside `/src`, you might have subdirectories for different modules of your application. For instance, you could have a `models` directory for your database models, a `routes` directory for your FastAPI `routes`, and a `services` directory for business logic.
- `/tests`: Keeping your tests separate from your application code is a good practice. It makes it easier to manage them and ensures that your production builds don't include test code.
- `/docs`: Documentation is crucial for any project. Whether it's API documentation, installation guides, or usage instructions, having a dedicated directory for documentation helps maintain clarity.

Note: We can use `Field` from `pydantic` for set validations.

For response we have:

Note: Also we have many responses

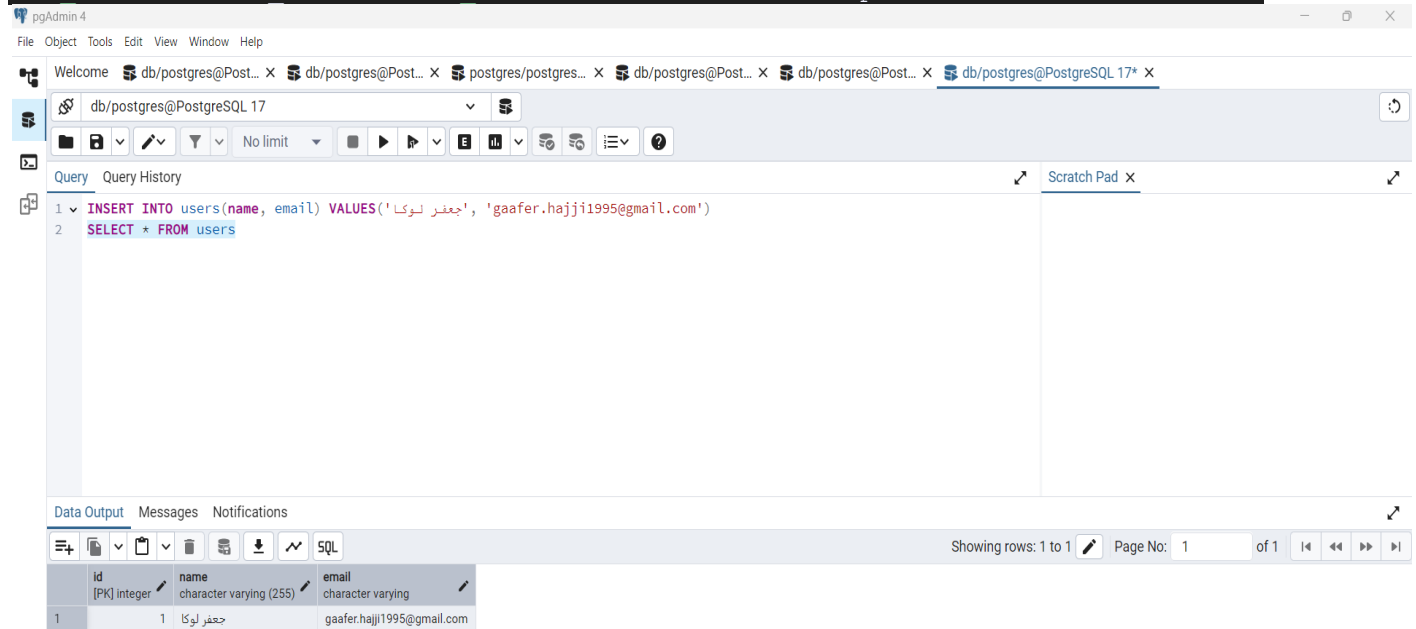
```
from fastapi.responses import JSONResponse, PlainTextResponse, FileResponse,
HTMLResponse, StreamingResponse
```

For supporting arabic encoding in postgresql, from postgres SQL Query Tool:

Note: in code we don't need to change anything after that

SET CLIENT_ENCODING TO 'UTF8';

```
CREATE DATABASE your_database_name WITH ENCODING = 'UTF8' LC_COLLATE =  
'ar_SA.UTF-8' LC_CTYPE = 'ar_SA.UTF-8' TEMPLATE = template0;
```



User schemas:

```
from pydantic import BaseModel, ConfigDict
```

```
class UserCreate(BaseModel):
```

```
    name: str
```

```
    email: str
```

```
class UserList(UserCreate):
```

```
    id: int
```

```
    model_config = ConfigDict(from_attributes=True)
```

```
-----
```

Users CRUD using async sqlalchemy:

```
from typing import Annotated, List
from fastapi import APIRouter, Depends, HTTPException

from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession

from db import get_async_db_session
from models.user import User

from schemas.user_schema import UserCreate, UserList

users_router = APIRouter()

@users_router.get('/', response_model=List[UserList])
async def get_all_users(db : Annotated[AsyncSession,
Depends(get_async_db_session) ]):

    result = await db.execute(select(User))
    users = result.scalars()
    # users = result.scalars().all() # This is also valid
    return users

@users_router.post('/', response_model=UserList, status_code=201)
async def create_new_user(db: Annotated[AsyncSession,
Depends(get_async_db_session)], user: UserCreate):

    t1 = User(
        email=user.email,
        name=user.name
    )

    db.add(t1)
    await db.commit()
    await db.refresh(t1)

    return t1

@users_router.get('/{id}', response_model=UserList)
async def get_user_by_id(id: int, db: Annotated[AsyncSession,
Depends(get_async_db_session)]):
    user = await db.execute(select(User).filter(User.id == id))
```

```

user = user.scalar_one_or_none()

if user is None:
    raise HTTPException(
        status_code=404,
        detail={
            "msg": "User not found",
        }
    )
return user

@users_router.put("/{user_id}", response_model=UserList)
async def update_user(user_id: int, user: UserCreate, db: Annotated[AsyncSession,
Depends(get_async_db_session)]):
    t1 = await db.execute(select(User).filter(User.id == user_id))

    t1 = t1.scalar_one_or_none()

    if t1 is None:
        raise HTTPException(
            status_code=404,
            detail={
                "msg": "User not found"
            }
        )

    print(t1)

    t1.name = user.name
    t1.email = user.email

    await db.commit()

    await db.refresh(t1)

    return t1

@users_router.delete("/{user_id}", status_code=204)
async def delete_user(user_id: int, db: Annotated[AsyncSession,
Depends(get_async_db_session)]):
    user = await db.execute(select(User).filter(User.id == user_id))

    user = user.scalar_one_or_none()

```

```

    if user is None:
        raise HTTPException(
            status_code=404,
            detail={
                "msg": "User not found"
            }
        )

    await db.delete(user)
    await db.commit()

    return { "result": "User deleted" }
*****

```

User validation using constr, field_validator, and EmailStr:

```

from pydantic import BaseModel, EmailStr, constr, field_validator

class User(BaseModel):
    name: constr(min_length=2, max_length=50) # type: ignore
    email: EmailStr
    age: int

    @field_validator('age')
    def validate_age(cls, value):
        if value < 18 or value > 100:
            raise ValueError("Age must be between 18 and 100")
        return value

class UserList(User):
    id: int
*****

```

Nosql Database with collection:

```

from pymongo import MongoClient

client = MongoClient()

db = client.jlokadb
-----

```

User collection:

```
from nosql_db import db

user_collection = db['users']
```

Schemas:

```
class UserCreate(BaseModel):
    name: str
    email: str

class NoSQLUserList(UserCreate):
    id: str
    model_config = ConfigDict(from_attributes=True)
```

Routes:

```
from typing import List

from fastapi import APIRouter, HTTPException

from bson import ObjectId

from schemas.user_schema import UserCreate, NoSQLUserList

from models.user_collection import user_collection

nosql_user_router = APIRouter()

@nosql_user_router.get("/", response_model=List[NoSQLUserList])
async def get_all_users():
    return [
        NoSQLUserList(
            id=str(user["_id"]),
            name=user["name"],
            email=user["email"],
        )
        for user in user_collection.find()
    ]

@nosql_user_router.post("/", response_model=NoSQLUserList)
async def create_user(user: UserCreate):
    t1 = user_collection.insert_one(user.model_dump(exclude_none=True))

    return NoSQLUserList(id=str(t1.inserted_id), **user.model_dump())

@nosql_user_router.get("/{user_id}", response_model=NoSQLUserList)
async def get_user(user_id: str):
    if not ObjectId.is_valid(user_id):
        raise HTTPException(status_code=400, detail={ "msg" : "user id not valid"
    })

    t1 = user_collection.find_one({ "_id": ObjectId(user_id) })
    if t1 is None:
        raise HTTPException(status_code=404, detail={ "msg": "No user found" })

    return NoSQLUserList(id=str(t1["_id"]), name=t1["name"], email=t1["email"])
*****
```

Simple upload file:

```
from fastapi import FastAPI, File, UploadFile
import aiofiles
from pathlib import Path

UPLOAD_FOLDER = Path("JLOKA")
UPLOAD_FOLDER.mkdir(exist_ok=True)

app = FastAPI()

@app.post("/")
async def getFile(file: UploadFile = File(...)):
    file_path = UPLOAD_FOLDER / file.filename # type: ignore

    async with aiofiles.open(file_path, 'wb') as f:
        content = await file.read()
        await f.write(content)

    return { "file": file.filename, "size": file.size / 1024 if file.size is not
None else 0}
*****
```

To hide the /token of OAuth2 from swagger we need to use:

```
from fastapi.openapi.utils import get_openapi
```

Then delete the path from new generated openapi schema:

```
del openapi_schema["paths"]["/token"]

app.openapi_schema = openapi_schema

return app.openapi_schema
```

Then we set the custom openapi to new app:

```
app = FastAPI(title="JLoka-01", description="JLoka-02 ", version="1.0.0",)
```

```
app.openapi = modified_openapi
```

```
*****
```

When we create model for MySQL DB, we must set the length of it for VARCHAR-Type:

```
from db import Base
from sqlalchemy import String
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
    __tablename__ = 'users'

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(255), nullable=False)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    password: Mapped[str] = mapped_column(String(255), nullable=False)

    def to_json(self):
        return {
            "id": self.id,
            "name": self.name,
            "email": self.email
        }
```

```
*****
```

```
import os
import datetime
from typing import Annotated
from fastapi import Depends
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from email_validator import validate_email, EmailNotValidError
from util.util import pwd_context
from models.users_model import User

oauth2_schema = OAuth2PasswordBearer(tokenUrl='/users/token')
```

```

async def authenticate_user(session: AsyncSession, email: str, password: str) ->
User|None:
    try:
        validate_email(email)

        user = (await session.execute(select(User).filter(User.email ==
email))).scalar_one_or_none()

        if not user:
            return None
        if not pwd_context.verify(password, user.password):
            return None
        return user
    except EmailNotValidError as e:
        return None

async def create_jwt_token(data: dict) -> str:
    payload = data.copy()

    expire_time = datetime.datetime.now(datetime.timezone.utc) \
        + datetime.timedelta(
            minutes=float(os.environ.get('ACCESS_TOKEN_EXPIRE_MINUTES', 30))
        )

    payload.update({'exp': expire_time})
    token = jwt.encode(payload, os.environ.get('SECRET_KEY', ''),
algorithm=os.environ.get('ALGORITHM', ''))
    return token

async def get_user_from_token(token: Annotated[str, Depends(oauth2_schema)],
session: AsyncSession) -> User | None:
    try:
        payload = jwt.decode(token, os.environ.get('SECRET_KEY', ''),
algorithm=os.environ.get('ALGORITHM', ''))
        email: str | None = payload.get("sub")
        if email is None:
            return None

        user = (await session.execute(select(User).filter(User.email ==
email))).scalar_one_or_none()
        return user

    except JWTError:
        return None

```

```

from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm, OAuth2PasswordBearer
from typing import Annotated
from sqlalchemy import select
from sqlalchemy.ext.asyncio import AsyncSession
from schemas.user_schema import UserSchemaRes, UserSchemaReq
from models.users_model import User
from db import get_async_db_session
from util.util import pwd_context
from security import authenticate_user, create_jwt_token, oauth2_schema,
get_user_from_token

user_router = APIRouter()

@user_router.post('/', response_model=UserSchemaRes)
async def create_user(user: UserSchemaReq, db: Annotated[AsyncSession,
Depends(get_async_db_session)]):
    print(f"The user Password is: {user.password}")
    user.password = pwd_context.hash(user.password)

    t2 = await db.execute(select(User).filter(User.email == user.email))
    t2 = t2.scalar_one_or_none()

    if t2 is not None:
        raise HTTPException(status_code=404, detail={ "msg" : "Email Exists
change it" })

    t1 = User(**user.model_dump())
    db.add(t1)
    await db.commit()
    await db.refresh(t1)

    return t1

@user_router.post('/token')
async def get_user_token(form_data:Annotated[OAuth2PasswordRequestForm,
Depends()], session: Annotated[AsyncSession, Depends(get_async_db_session)] ):
    user = await authenticate_user(session, form_data.username,
form_data.password)
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Incorrect username or password 3")

    token = await create_jwt_token(data= { 'sub': user.email})

```

```
    return {  
        "token": token,  
        "type": "bearer",  
    }
```

```
@user_router.get('/me')  
async def get_user_profile(token: Annotated[str, Depends(oauth2_schema)],  
    session: Annotated[AsyncSession, Depends(get_async_db_session)]):  
    user = await get_user_from_token(token, session)  
    if not user:  
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED)  
    return user.to_json()
```